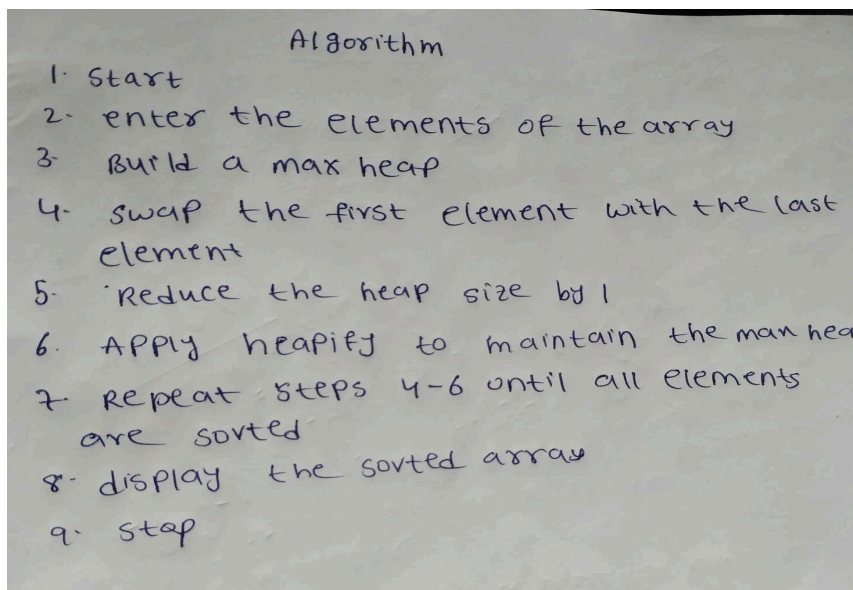


Practical 1

Aim: Implementation of max-heap sort algorithm.

Algorithms:



CODE:

```
#include <iostream>

using namespace std;

// Function to maintain Max Heap

void heapify(int arr[], int n, int i)

{

    int largest = i;

    int left = 2 * i + 1;

    int right = 2 * i + 2;

    // Check if left child is larger
```

```
if (left < n && arr[left] > arr[largest])

largest = left;

// Check if right child is larger

if (right < n && arr[right] > arr[largest])

largest = right;

// If largest is not root

if (largest != i)

{

swap(arr[i], arr[largest]);

heapify(arr, n, largest);

}

}

void heapSort(int arr[], int n)

{

for (int i = n / 2 - 1; i >= 0; i--)

{

heapify(arr, n, i);

}

for (int i = n - 1; i > 0; i--)

{

swap(arr[0], arr[i]);

heapify(arr, i, 0);

}

}

int main()
```

```
{  
int n;  
cout << "Enter number of elements: ";  
cin >> n;  
int* arr = new int[n];  
cout << "Enter " << n << " elements: ";  
for (int i = 0; i < n; i++)  
{  
cin >> arr[i];  
}  
heapSort(arr, n);  
cout << "Sorted array: ";  
for (int i = 0; i < n; i++)  
{  
cout << arr[i] << " ";  
}  
cout << endl;  
delete[] arr;  
return 0;  
}
```



OUTPUT:

```
Enter number of elements: 4
Enter 4 elements: 1
4
45
3
Sorted array: 1 3 4 45

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Time complexity

1. build max heap

for $(i = n/2 - 1; i >= 0; i--)$

{

 heapify(arr, n, i);

}

3

Building the heap takes $O(n)$

inside heapify

 if (left < n)

 if (right < n)

Height of a binary heap:

$\log_2(n)$

heapify = $O(\log n)$

$\therefore$ heapify = $n \times O(\log n)$

 = $O(n \log n)$

So. best case, Average case, worst case

is all same for time complexity